Social Dynamics Simulator: Mathematical Overview

Contents

1 Notation **2**

2 Core Dynamics: Per-Dimension Max-Preference Physics **2**
- 2.1 Algorithm . . . 2
- 2.2 Worked Example: 3 Particles, $K = 3$ . . . 2

3 Best Neighbor Selection Modes **3**
- 3.1 Mode 0: Default (highest value) . . . 3
- 3.2 Mode 1: Max Magnitude . . . 4
- 3.3 Mode 2: Same-Sign Max Magnitude . . . 4

4 Inner-Product Average Mode **4**

5 Signal / Response Split **4**
- 5.1 Compatibility . . . 4

6 Repulsion **4**

7 Social Learning **5**
- 7.1 Quiet-Dimension Differentiation . . . 5

8 Spatial Memory Field **5**
- 8.1 Read (before physics) . . . 5
- 8.2 Write (after physics) . . . 5
- 8.3 Decay . . . 5
- 8.4 Blur (optional) . . . 5

9 Neighbor Finding **5**
- 9.1 KNN (K-Nearest Neighbors) . . . 5
- 9.2 Radius . . . 6
- 9.3 KNN + Radius . . . 6

10 Grid Field Approximations **6**
- 10.1 Smooth Grid Field . . . 6
- 10.2 Grid Max Field . . . 6

11 Force Landscape **6**

12 Shadow Simulation **7**

13 Precision Controls **7**
13.1 Dtype Selection 7
13.2 Mantissa Truncation 7
13.3 Level Quantization 7

14 Direction Memory **7**

15 Full Update Rule **8**

1 Notation

Symbol	Definition
N	Number of particles
K	Number of preference dimensions per particle
i, j	Particle indices, $i, j \in \{1, \ldots, N\}$
d	Preference dimension index, $d \in \{1, \ldots, K\}$
$\mathbf{x}_i \in [0, L)^2$	Position of particle i (toroidal domain, $L = 1$)
$\mathbf{p}_i \in [-1, 1]^K$	Signal (preference) vector of particle i
$\mathbf{r}_i \in [-1, 1]^K$	Response vector of particle i
$\mathcal{N}(i)$	Neighbor set of particle i
$j_d^*(i)$	Best neighbor of i in dimension d
η	Step size
L	Domain size (default 1.0, toroidal)
$\Delta(\mathbf{a}, \mathbf{b})$	Periodic displacement: $\mathbf{b} - \mathbf{a} - L \cdot \mathrm{round}\left(\frac{\mathbf{b}-\mathbf{a}}{L}\right)$

2 Core Dynamics: Per-Dimension Max-Preference Physics

The fundamental interaction rule: each particle moves toward the neighbors that have the strongest preference values, weighted by its own preferences.

2.1 Algorithm

For each particle i at each time step:

1. **Find neighbors**: Determine $\mathcal{N}(i)$, the set of nearby particles (via KNN or radius search).
2. **Per-dimension best neighbor**: For each dimension $d \in \{1, \ldots, K\}$, find the neighbor with the highest signal value:
$$j_d^*(i) = \arg\max_{j \in \mathcal{N}(i)} p_j^{(d)} \tag{1}$$
3. **Direction**: Compute the unit direction from i toward each best neighbor (periodic):
$$\hat{\mathbf{u}}_d(i) = \frac{\Delta(\mathbf{x}_i,\, \mathbf{x}_{j_d^*})}{\|\Delta(\mathbf{x}_i,\, \mathbf{x}_{j_d^*})\|} \tag{2}$$
4. **Compatibility**: The weight for dimension d is the product of the particle's own response and the best neighbor's signal:
$$w_d(i) = r_i^{(d)} \cdot p_{j_d^*}^{(d)} \tag{3}$$
5. **Movement**: Sum the weighted directions across all dimensions:
$$\mathbf{v}_i = \sum_{d=1}^{K} w_d(i) \cdot \hat{\mathbf{u}}_d(i) \tag{4}$$
6. **Position update**:
$$\mathbf{x}_i \leftarrow (\mathbf{x}_i + \eta \cdot \mathbf{v}_i) \bmod L \tag{5}$$

2.2 Worked Example: 3 Particles, $K = 3$

Consider three particles A, B, C with positions and preferences:

Particle	Position	Signal $\mathbf{p}$	Response $\mathbf{r}$
A	$(0.3, 0.5)$	$(0.8, -0.3, 0.5)$	$(0.8, -0.3, 0.5)$
B	$(0.6, 0.4)$	$(0.2, 0.9, -0.7)$	$(0.2, 0.9, -0.7)$
C	$(0.5, 0.8)$	$(-0.4, 0.6, 0.9)$	$(-0.4, 0.6, 0.9)$

Without signal/response split, $\mathbf{r}_i = \mathbf{p}_i$.

Step 1: All particles are neighbors of each other ($N = 3$, $K_{\text{nn}} = 2$).
Step 2: For particle A, find the best neighbor per dimension:

Dim d	Best neighbor	$p_{j^*}^{(d)}$	Why
1	B ($p^{(1)} = 0.2$)	0.2	$\max(0.2, -0.4) = 0.2$
2	B ($p^{(2)} = 0.9$)	0.9	$\max(0.9, 0.6) = 0.9$
3	C ($p^{(3)} = 0.9$)	0.9	$\max(-0.7, 0.9) = 0.9$

Step 3: Directions from A toward each best neighbor:

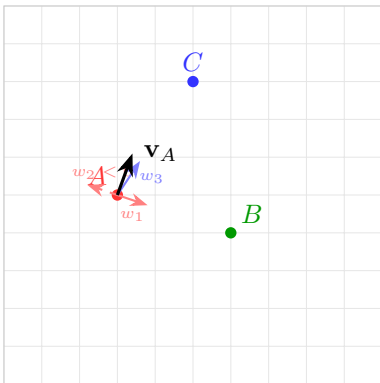
$$\hat{\mathbf{u}}_1 = \hat{\mathbf{u}}_2 = \text{normalize}(B - A) = \text{normalize}(0.3, -0.1) \approx (0.949, -0.316)$$
$$\hat{\mathbf{u}}_3 = \text{normalize}(C - A) = \text{normalize}(0.2, 0.3) \approx (0.555, 0.832)$$

Step 4: Compatibility weights (response $\times$ signal):

$$w_1 = r_A^{(1)} \cdot p_B^{(1)} = 0.8 \times 0.2 = 0.16$$
$$w_2 = r_A^{(2)} \cdot p_B^{(2)} = -0.3 \times 0.9 = -0.27$$
$$w_3 = r_A^{(3)} \cdot p_C^{(3)} = 0.5 \times 0.9 = 0.45$$

Note: $w_2 < 0$ means particle A moves *away* from B on dimension 2 (opposite preferences).
Step 5: Total movement:

$$\begin{aligned}\mathbf{v}_A &= 0.16 \cdot (0.949, -0.316) + (-0.27) \cdot (0.949, -0.316) + 0.45 \cdot (0.555, 0.832) \\ &= (0.152, -0.051) + (-0.256, 0.085) + (0.250, 0.374) \\ &= (0.146, 0.409)\end{aligned}$$



Key insight: Dimensions 1 and 2 both point toward B, but with opposite signs. Dimension 3 points toward C. The resultant movement depends on the *balance* of these competing forces.

3 Best Neighbor Selection Modes

The selection of $j_d^*(i)$ can be modified:

3.1 Mode 0: Default (highest value)

$$j_d^*(i) = \arg\max_{j \in \mathcal{N}(i)} p_j^{(d)} \tag{6}$$

Selects the neighbor with the highest raw preference. Positive values are preferred over negative.

3.2 Mode 1: Max Magnitude

$$j_d^*(i) = \arg\max_{j \in \mathcal{N}(i)} |p_j^{(d)}| \tag{7}$$

Selects the neighbor with the strongest preference regardless of sign.

3.3 Mode 2: Same-Sign Max Magnitude

$$j_d^*(i) = \arg \max_{\substack{j \in \mathcal{N}(i) \\ \text{sign}(p_j^{(d)}) = \text{sign}(p_i^{(d)})}} |p_j^{(d)}| \tag{8}$$

Only considers neighbors whose preference has the same sign as particle i in dimension d. If no same-sign neighbor exists, dimension d contributes zero to the movement.

4 Inner-Product Average Mode

An alternative to the per-dimension best-neighbor selection. Instead of finding one best neighbor per dimension, this mode weights *all* neighbors by the full inner product of their preference vectors:

$$\mathbf{v}_i = \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} \frac{\mathbf{r}_i \cdot \mathbf{p}_j}{K} \cdot \hat{\mathbf{u}}_{ij} \tag{9}$$

where $\hat{\mathbf{u}}_{ij} = \Delta(\mathbf{x}_i, \mathbf{x}_j)/\|\Delta(\mathbf{x}_i, \mathbf{x}_j)\|$.

Particles with aligned preferences attract ($\mathbf{r}_i \cdot \mathbf{p}_j > 0$), while misaligned particles repel.

5 Signal / Response Split

By default, $\mathbf{r}_i = \mathbf{p}_i$. The signal/response split separates these into independent vectors:

- **Signal** $\mathbf{p}_i$: what particle i broadcasts. Used by other particles for neighbor selection ($\arg\max p_j^{(d)}$).
- **Response** $\mathbf{r}_i$: how particle i reacts. Used in the compatibility weight ($w_d = r_i^{(d)} \cdot p_{j^*}^{(d)}$).

This breaks the symmetry: a particle can listen for signals it doesn't broadcast, or broadcast signals it doesn't respond to.

5.1 Compatibility

With the split:

$$w_d(i) = r_i^{(d)} \cdot p_{j_d^*}^{(d)} \tag{10}$$

The selection j_d^* always uses the **signal** vector of the neighbors. The weighting uses the **response** of particle i and the **signal** of the selected neighbor.

6 Repulsion

An optional repulsive force pushes particles away from close neighbors:

$$\mathbf{v}_i^{\text{rep}} = -\frac{\rho}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} \frac{\hat{\mathbf{u}}_{ij}}{\|\Delta(\mathbf{x}_i, \mathbf{x}_j)\|} \tag{11}$$

where ρ is the repulsion strength. The $1/r$ scaling creates stronger repulsion at close range.

7 Social Learning

Preferences evolve over time via social learning from neighbors:

$$\mathbf{p}_i \leftarrow (1-s)\cdot \mathbf{p}_i + s\cdot \bar{\mathbf{p}}_{\mathcal{N}(i)} \tag{12}$$

where $\bar{\mathbf{p}}_{\mathcal{N}(i)} = \frac{1}{|\mathcal{N}(i)|}\sum_{j\in\mathcal{N}(i)} \mathbf{p}_j$ is the neighbor mean preference.

- $s > 0$: **Conformity** — preferences move toward the local mean.
- $s < 0$: **Differentiation** — preferences move away from the local mean.
- $s = 0$: Preferences are fixed.

7.1 Quiet-Dimension Differentiation

Instead of uniform social learning across all dimensions, scale the learning rate inversely by each dimension's contribution to movement:

$$s_d(i) = s\cdot\left(1 - \frac{|w_d(i)|}{\max_{d'}|w_{d'}(i)|}\right) \tag{13}$$

Dimensions that contribute most to movement (high $|w_d|$) are preserved. Dimensions that contribute least (low $|w_d|$) are differentiated most. This allows particles to specialize on "quiet" dimensions without disrupting the primary dynamics.

8 Spatial Memory Field

A persistent grid field $M \in \mathbb{R}^{G\times G\times K}$ creates feedback between dynamics and environment.

8.1 Read (before physics)

$$\tilde{p}_i^{(d)} = p_i^{(d)}\cdot(1+\alpha\cdot M(\mathrm{cell}(\mathbf{x}_i), d)) \tag{14}$$

Clipped to $[-1, 1]$. The field amplifies aligned preferences and dampens opposed ones.

8.2 Write (after physics)

$$M(\mathrm{cell}(\mathbf{x}_i), d) \mathrel{+}= \beta\cdot p_i^{(d)} \tag{15}$$

8.3 Decay

$$M \leftarrow \gamma\cdot M \tag{16}$$

8.4 Blur (optional)

$$M^{(d)} \leftarrow M^{(d)} * \mathcal{G}(\sigma) \tag{17}$$

Gaussian diffusion with periodic wrapping.

See `spatial_memory.pdf` for detailed analysis of regimes and fixed points.

9 Neighbor Finding

9.1 KNN (K-Nearest Neighbors)

Find the K_{nn} closest particles by periodic Euclidean distance. Implemented via:

- **cKDTree**: $O(N\log N)$ build, $O(K_{\mathrm{nn}}\log N)$ per query. Best for KNN.
- **Spatial Hash Grid**: $O(N)$ build, ring-expansion query. Best for radius queries.

9.2 Radius

Find all particles within distance R. Neighbor count varies per particle (can be very large in clusters).

9.3 KNN + Radius

Find K_{nn} nearest, then filter to those within radius R. Bounded cost with distance cutoff.

10 Grid Field Approximations

For large N, pairwise neighbor search becomes the bottleneck. Grid-based methods approximate the physics at $O(N + G^2)$ cost.

10.1 Smooth Grid Field

Deposit preferences onto a $G \times G$ grid via bilinear interpolation, Gaussian-smooth, compute gradients:

$$\mathbf{v}_i = \sum_{d=1}^{K} r_i^{(d)} \cdot \nabla F^{(d)}(\mathbf{x}_i) \tag{18}$$

where $F^{(d)}$ is the smoothed signal field for dimension d.

10.2 Grid Max Field

Approximates the per-dimension best-neighbor physics on a grid:

1. **Deposit**: For each cell, store the max signal value per dim and the source particle's position.
2. **Propagate**: Iterative 3×3 max-pool (8-connected) spreads max values to neighboring cells, carrying source positions. After P passes, each cell knows the best particle within Chebyshev distance P.
3. **Movement**: Each particle reads its cell's per-dim max and source position:

$$\mathbf{v}_i = \sum_{d=1}^{K} r_i^{(d)} \cdot M_d^{\max}(\text{cell}(\mathbf{x}_i)) \cdot \hat{\mathbf{u}}_d^{\max}(\text{cell}(\mathbf{x}_i)) \tag{19}$$

The GPU variant uses fused `max_pool2d` with `return_indices` for single-kernel propagation.

11 Force Landscape

The force landscape answers: "at each point in space, what preference vector would produce the maximum movement?"

For a probe at position $\mathbf{x}$, the movement is linear in the response vector:

$$\mathbf{v}(\mathbf{x}, \mathbf{r}) = \mathbf{M}(\mathbf{x}) \cdot \mathbf{r} \tag{20}$$

where $\mathbf{M}(\mathbf{x})$ is the $2 \times K$ matrix with columns $p_{j_d^*}^{(d)} \cdot \hat{\mathbf{u}}_d$, built from the actual KNN neighbors at position $\mathbf{x}$.

The optimal response is found by exhaustive search over $\{-1, 0, +1\}^K$ (3^K candidates):

$$\mathbf{r}^* = \arg \max_{\mathbf{r} \in \{-1,0,+1\}^K} \|\mathbf{M}(\mathbf{x}) \cdot \mathbf{r}\| \tag{21}$$

Three visualization modes:

- **Max Pref**: the per-dim max signal values as RGB, brightness by force magnitude.

- **Optimal Pref**: the optimal $\mathbf{r}^*$ as RGB.
- **Direction**: angle of $\mathbf{M} \cdot \mathbf{r}^*$ as HSV hue, magnitude as brightness.

An exponential moving variance mode shows temporal stability:

$$\mu_t = \gamma \mu_{t-1} + (1-\gamma) x_t \tag{22}$$

$$\sigma_t^2 = \gamma \left(\sigma_{t-1}^2 + (1-\gamma)(x_t - \mu_{t-1})^2 \right) \tag{23}$$

12 Shadow Simulation

A deep copy of the simulation with LSB-perturbed positions tracks chaotic sensitivity:

$$\tilde{\mathbf{x}}_i = \mathbf{x}_i \oplus \delta_i \tag{24}$$

where δ_i randomizes the lowest B bits of the IEEE 754 mantissa. For $B = 1$, the perturbation is $\sim 10^{-16}$ (one ULP).

Divergence is measured as RMS periodic distance:

$$D(t) = \sqrt{\frac{1}{N} \sum_{i=1}^{N} \|\Delta(\mathbf{x}_i(t),\, \tilde{\mathbf{x}}_i(t))\|^2} \tag{25}$$

Exponential growth of $D(t)$ indicates chaotic dynamics.

13 Precision Controls

13.1 Dtype Selection

- Positions: `float32` or `float64`
- Preferences: `float16` (10-bit mantissa), `float32` (23-bit), or `float64` (52-bit)
- Physics compute (PyTorch): `float16`, `bfloat16`, `float32`, `float64`

13.2 Mantissa Truncation

After each step, zero the lowest $(M - B)$ mantissa bits:

$$x_{\text{trunc}} = \text{int}(x) \gg (M - B) \ll (M - B) \tag{26}$$

where M is the full mantissa width and B is the target bits.

13.3 Level Quantization

Snap preferences to Q evenly spaced values in $[-1, 1]$:

$$p_{\text{quant}} = \frac{\text{round}\left(\frac{p+1}{2} \cdot (Q-1)\right)}{Q-1} \cdot 2 - 1 \tag{27}$$

Special case $Q = 2$: binary $\{-1, +1\}$ via $\text{sign}(p)$.

14 Direction Memory

An exponential moving average (EMA) of the per-dimension direction vectors:

$$\mathbf{d}_i^{(k)} \leftarrow \lambda \cdot \mathbf{d}_i^{(k)} + (1-\lambda) \cdot \hat{\mathbf{u}}_k(i) \tag{28}$$

where λ is the direction memory parameter. The movement then uses $\mathbf{d}_i^{(k)}$ instead of $\hat{\mathbf{u}}_k(i)$, giving particles momentum that persists across neighbor changes.

15 Full Update Rule

Combining all components, the complete per-step update for particle i is:

1. Read memory field: $\tilde{\mathbf{p}}_i = \mathbf{p}_i \cdot (1 + \alpha \cdot M(\text{cell}(\mathbf{x}_i)))$
2. Find neighbors $\mathcal{N}(i)$
3. For each dim d: find j_d^*, compute $\hat{\mathbf{u}}_d$, update direction memory $\mathbf{d}_d$
4. Compute compatibility: $w_d = \tilde{r}_i^{(d)} \cdot \tilde{p}_{j_d^*}^{(d)}$
5. Optionally multiply by inner product: $w_d \times= \frac{\tilde{\mathbf{r}}_i \cdot \tilde{\mathbf{p}}_{j_d^*}}{K}$
6. Movement: $\mathbf{v}_i = \sum_d w_d \cdot \mathbf{d}_d + \mathbf{v}_i^{\text{rep}}$
7. Update position: $\mathbf{x}_i \leftarrow (\mathbf{x}_i + \eta \cdot \mathbf{v}_i) \bmod L$
8. Social learning: $\mathbf{p}_i \leftarrow (1 - s_d) \cdot \mathbf{p}_i + s_d \cdot \bar{\mathbf{p}}_{\mathcal{N}(i)}$
9. Write memory field: $M(\text{cell}(\mathbf{x}_i)) += \beta \cdot \mathbf{p}_i$
10. Decay memory: $M \leftarrow \gamma \cdot M$; optionally blur M
11. Optionally truncate/quantize $\mathbf{x}_i$ and $\mathbf{p}_i$